

Advanced Programming Techniques

fbeit

FACHBEREICH ELEKTROTECHNIK
UND INFORMATIONSTECHNIK

Prof. Dr. Michael Lipp

C++ Libraries (2)

fbeit

FACHBEREICH ELEKTROTECHNIK
UND INFORMATIONSTECHNIK

Container type “set” (1)

- A container that contains each element only once

```
set<int> mySet = { 1, 2, 3, 4, 5, 6, 7 };  
mySet.insert(5);  
for (auto value: mySet) {  
    cout << value << endl;  
}
```

- Elements must implement **operator<**
 - Used to determine equality
 - Alternative: provide compare method in map constructor

Container type “set” (2)

- Note additional functions in **<algorithm>**
 - set_union
 - set_intersection
 - set_difference

Container type map

- Often a “mapping” (a.k.a. association) is needed
 - A mapping associates a key with a value
 - Examples:
 - Phone book (name → number)
 - Page index (keyword → page number)
 - Student database (Matrikel number → student data)
- Provided by STL type **map**
 - Requires two type parameters (one for key's type, one for associated value's type)
 - Type of key must implement `operator<`

Container type map

- Functions

- Inserting mappings (associations)

- Use overloaded operator[] (usually preferred)

```
map<string, int> nameToValue = {{ "one", 1}, {"two", 2}};  
nameToValue["zero"] = 0;
```

- Use type std::pair

```
nameToValue.insert(pair<string, int>("zero", 0));
```

- Looking up a value associated with a key

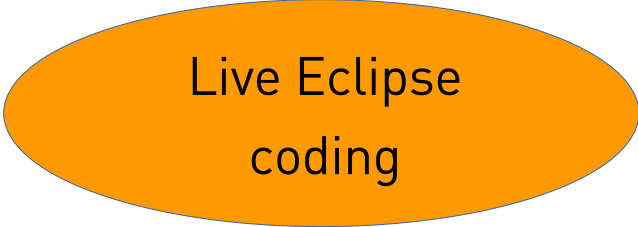
- cout << nameToValue["zero"] << endl;
 - cout << nameToValue.at("zero") << endl;

- The “usual” container functions (size, empty etc.)

- See <http://www.cplusplus.com/reference/map/map/>

Usage example

- (Basic) Student database



Live Eclipse
coding

fbeit

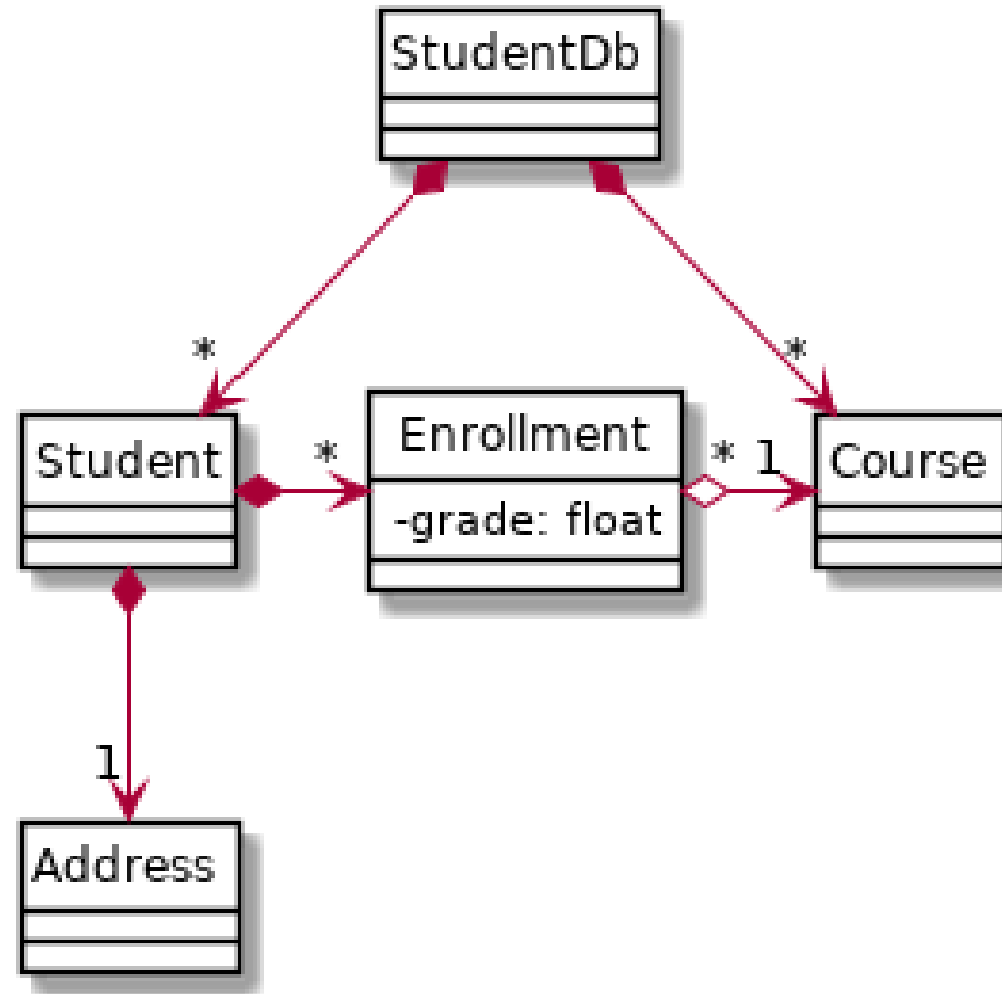
FACHBEREICH ELEKTROTECHNIK
UND INFORMATIONSTECHNIK

Student Database

Classes for a simple Student Database



Class relationships



Types of relationships

- **Association**

- Two classes (rather, instances of the classes) know about each other
 - In class diagram: line between classes

- **Aggregation (weak part-whole relationship)**

- An instance of a class A contains/consists of instances of another class B (or: instances of B are part of an instance of A)
 - In class diagram: line with unfilled diamond shape at containing class

- **Composition (strong part-whole relationship)**

- Like aggregation, but if the containing object is deleted, the contained objects (parts) are also deleted
 - In class diagram: line with filled diamond shape at containing class

Additional information

- **Relationships in UML diagram can be augmented with additional information**
 - Number of related objects (denoted at the class of the related objects)
 - Fixed number, e.g. “5”: exactly 5 related objects
 - Range, e.g. “1..3”: at least 1, at most 3 related objects (note: two dots in ellipses)
 - Unlimited: “*” (meaning 0 to many, can also be used as upper limit of range: “1..*”)
 - Direction (open arrowhead at one end)
 - Relationship can be navigated only from one participating object (e.g. of type A) to the other participating object (e.g. of type B)
 - Implementation: A has a data member of type B or a pointer to type B
 - B does not have such a data member